

Task 8: SQL Injection Practical Exploitation

Tools Used

- DVWA (Damn Vulnerable Web Application)
- SQLMap
- Apache Web Server
- MySQL
- Ubuntu Linux

1) Identify injectable parameters

I opened the DVWA SQL Injection page and entered a user ID.
The URL became:

<http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit>

Here, the parameter **id** is taken from user input and used directly in an SQL query.
This made it vulnerable to SQL Injection.

2) Run SQLMap

I copied the vulnerable URL and used my PHP session ID (PHPSESSID) to run SQLMap so that it could access the DVWA page.

SQLMap confirmed that the **id** parameter was injectable and the backend database was **MySQL**.

This proved that the web application was vulnerable.

3) Extract Database Names

Using SQLMap, I listed all the databases on the server.
The following databases were found:

- dvwa
- information_schema
- performance_schema

This showed that the attacker could see all databases stored on the server.

```
[18:16:58] [WARNING] reflective value(s) found and filtering out
available databases [3]:
[*] dvwa
[*] information_schema
[*] performance_schema
```

4) Extract Tables

I focused on the **dvwa** database and extracted the tables.

SQLMap showed that the database contained a table named:

- **users**

This table stores login details of users.

```
[18:19:29] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[1 table]
+-----+
| users |
+-----+
```

5) Extract User Data

I then dumped the **users** table.

SQLMap extracted usernames and password hashes from the database.

After that, SQLMap used a dictionary attack to crack the password hashes.

The cracked passwords included weak passwords like:

- password
- abc123
- letmein
- charley

This means the real login credentials of users were successfully recovered.

```

Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | user | avatar | password | last_name | first_name | last_login | failed_login |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | /DVWA/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin | admin | 2026-01-27 18:02:14 | 0 |
| 2 | gordonb | /DVWA/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123) | Brown | Gordon | 2026-01-27 18:02:14 | 0 |
| 3 | 1337 | /DVWA/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | Me | Hack | 2026-01-27 18:02:14 | 0 |
| 4 | pablo | /DVWA/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | Picasso | Pablo | 2026-01-27 18:02:14 | 0 |
| 5 | smithy | /DVWA/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob | 2026-01-27 18:02:14 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+

[18:24:42] [INFO] table 'dvwa.users' dumped to CSV file '/home/mounika/.local/share/sqlmap/output/localhost/dump/dvwa/users.csv'
[18:24:42] [INFO] fetched data logged to text files under '/home/mounika/.local/share/sqlmap/output/localhost'
[18:24:42] [WARNING] your sqlmap version is outdated

[*] ending @ 18:24:42 /2026-01-27/

```

6) Impact Analysis

This attack shows that if a web application is vulnerable to SQL Injection:

- Attackers can read all database data
- Usernames and passwords can be stolen
- Attackers can log in as admin
- Data can be modified or deleted
- The whole website can be taken over

This makes SQL Injection one of the most dangerous web vulnerabilities.

7) Attack Flow

The attack happened in the following steps:

1. Open the vulnerable SQL Injection page
2. Identify the `id` parameter
3. Run SQLMap on the URL
4. Extract database names
5. Extract tables from the dvwa database
6. Dump the users table
7. Crack password hashes

This shows how a simple input box can lead to full database compromise.

8) Security Fixes

To prevent SQL Injection:

- Use **prepared statements (parameterized queries)**
- Validate and sanitize all user inputs
- Do not display database errors
- Use least-privilege database accounts
- Use Web Application Firewalls (WAF)

These methods can stop attackers from injecting SQL code.

9) Conclusion

This task helped me understand how SQL Injection works in real web applications. By using SQLMap on DVWA, I was able to extract databases, tables, and user credentials. I also learned why secure coding practices are very important to protect web applications from attacks.